

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №9.

Тема: «Обработка исключительных ситуаций»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

1. Создать класс-контейнер «Множество», который хранит элементы типа `int`.
2. Перегрузить для этого класса следующие операции:
 - `[]` — доступ к элементу по индексу;
 - `()` — получение размера множества;
 - (бинарный) — разность двух множеств;
 - (унарный) — удаление элемента из множества.
3. Предусмотреть обработку исключительных ситуаций (ошибок).
4. Сделать так, чтобы при ошибках программа генерировала исключения.

Анализ задачи

1. Создаём класс `Set` для хранения множества целых чисел. Добавляем приватные поля: динамический массив `elements`, размер `size`, вместимость `capacity`.
2. Добавляем приватный метод `contains` для проверки наличия элемента в множестве.
3. Добавляем приватный метод `ensureCapacity` для автоматического расширения массива при необходимости (при достижении `capacity` увеличиваем вдвое, начальная вместимость 10).
4. Создаём конструкторы: по умолчанию (`size=0`, `capacity=10`), с инициализацией от массива (добавляем только уникальные элементы), копирования.
5. Создаём деструктор для освобождения памяти.
6. Перегружаем оператор присваивания `=` с глубоким копированием.
7. Перегружаем оператор `[]` для доступа по индексу с проверкой границ (при ошибке выводим сообщение и генерируем исключение 1).
8. Перегружаем оператор `()` для возврата размера множества.
9. Перегружаем оператор `-` для разности множеств: `Set - Set` возвращает новое множество с элементами из первого, отсутствующими во втором. `Set - int` возвращает новое множество без указанного элемента.
10. Создаём метод `removeElement` для удаления элемента, вводимого пользователем с клавиатуры (при пустом множестве - исключение 2, при отсутствии элемента - исключение 3).
11. Создаём метод `add` для добавления элемента. Проверяем на дубликаты (при наличии - исключение 4). Добавляем элемент, затем сортируем массив пузырьком по возрастанию.

12. Создаём метод print для вывода множества в формате { элементы } с указанием размера.

13. Создаём метод getSize для получения размера.

14. В функции main: создаём множества A и B, добавляем в них элементы. Демонстрируем работу оператора () для получения размера. Демонстрируем работу оператора [] для доступа по индексу с обработкой исключений. Демонстрируем оператор - для разности множеств A-B и B-A. Демонстрируем оператор - для удаления элемента по значению. Демонстрируем обработку исключений при добавлении дубликата, при доступе за границы массива. Демонстрируем работу с пустым множеством.

Код

```
#include <iostream>
#include <cstring>
#include <clocale>

using namespace std;

class Set {
private:
    int* elements;
    int size;
    int capacity;

    bool contains(int value) const {
        for (int i = 0; i < size; i++) {
            if (elements[i] == value) {
                return true;
            }
        }
        return false;
    }

    void ensureCapacity() {
        if (size >= capacity) {
            capacity = (capacity == 0) ? 10 : capacity * 2;
            int* newElements = new int[capacity];
            for (int i = 0; i < size; i++) {
                newElements[i] = elements[i];
            }
            delete[] elements;
            elements = newElements;
        }
    }

public:
    Set() : size(0), capacity(10) {
        elements = new int[capacity];
    }

    Set(const int* init, int initSize) : size(0), capacity(10) {
        elements = new int[capacity];
        for (int i = 0; i < initSize; i++) {
            if (!contains(init[i])) {
                add(init[i]);
            }
        }
    }
}
```

```

Set(const Set& other) : size(other.size), capacity(other.capacity) {
    elements = new int[capacity];
    for (int i = 0; i < size; i++) {
        elements[i] = other.elements[i];
    }
}

~Set() {
    delete[] elements;
}

Set& operator=(const Set& other) {
    if (this != &other) {
        delete[] elements;
        size = other.size;
        capacity = other.capacity;
        elements = new int[capacity];
        for (int i = 0; i < size; i++) {
            elements[i] = other.elements[i];
        }
    }
    return *this;
}

int& operator[](int index) {
    if (index < 0 || index >= size) {
        cout << "Ошибка: индекс вне диапазона. Размер множества: " << size
              << ", запрошен индекс: " << index << endl;
        throw 1;
    }
    return elements[index];
}

const int& operator[](int index) const {
    if (index < 0 || index >= size) {
        cout << "Ошибка: индекс вне диапазона. Размер множества: " << size
              << ", запрошен индекс: " << index << endl;
        throw 1;
    }
    return elements[index];
}

int operator()() const {
    return size;
}

Set operator-(const Set& other) const {
    Set result;
    for (int i = 0; i < size; i++) {
        if (!other.contains(elements[i])) {
            result.add(elements[i]);
        }
    }
    return result;
}

Set operator-(int value) const {
    Set result(*this);
    for (int i = 0; i < result.size; i++) {
        if (result.elements[i] == value) {
            for (int j = i; j < result.size - 1; j++) {
                result.elements[j] = result.elements[j + 1];
            }
            result.size--;
            break;
        }
    }
    return result;
}

```

```

bool removeElement() {
    if (size == 0) {
        cout << "Ошибка: невозможно удалить элемент из пустого множества" << endl;
        throw 2;
    }

    cout << "Введите элемент для удаления: ";
    int value;
    cin >> value;

    for (int i = 0; i < size; i++) {
        if (elements[i] == value) {
            for (int j = i; j < size - 1; j++) {
                elements[j] = elements[j + 1];
            }
            size--;
            cout << "Элемент " << value << " удален." << endl;
            return true;
        }
    }

    cout << "Ошибка: элемент " << value << " не найден во множестве" << endl;
    throw 3;
}

void add(int value) {
    if (contains(value)) {
        cout << "Ошибка: элемент " << value << " уже существует во множестве" <<
endl;
        throw 4;
    }
    ensureCapacity();
    elements[size++] = value;

    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (elements[j] > elements[j + 1]) {
                int temp = elements[j];
                elements[j] = elements[j + 1];
                elements[j + 1] = temp;
            }
        }
    }
}

void print() const {
    cout << "{ ";
    for (int i = 0; i < size; i++) {
        cout << elements[i];
        if (i < size - 1) cout << ", ";
    }
    cout << " } (размер: " << size << ")" << endl;
}

int getSize() const {
    return size;
}
};

int main() {
    setlocale(LC_ALL, "Russian");

    cout << "ТЕСТИРОВАНИЕ КЛАССА МНОЖЕСТВО\n" << endl;

    cout << "Тест 1: Создание множеств" << endl;
    Set A;
    Set B;

```

```

try {
    A.add(5);
    A.add(3);
    A.add(8);
    A.add(1);
    A.add(10);

    B.add(5);
    B.add(8);
    B.add(2);
    B.add(7);

    cout << "Множество A: ";
    A.print();
    cout << "Множество B: ";
    B.print();
}
catch (int e) {
    cout << "Исключение с кодом: " << e << endl;
}

cout << "\nТест 2: Оператор () - определение размера" << endl;
cout << "Размер A: " << A() << endl;
cout << "Размер B: " << B() << endl;

cout << "\nТест 3: Оператор [] - доступ по индексу" << endl;
try {
    cout << "A[0] = " << A[0] << endl;
    cout << "A[1] = " << A[1] << endl;
    cout << "A[2] = " << A[2] << endl;
    cout << "A[3] = " << A[3] << endl;
    cout << "A[4] = " << A[4] << endl;
}
catch (int e) {
    cout << "Исключение с кодом: " << e << endl;
}

cout << "\nТест 4: Оператор - (разность множеств)" << endl;
Set C = A - B;
cout << "A - B = ";
C.print();

Set D = B - A;
cout << "B - A = ";
D.print();

cout << "\nТест 5: Оператор - (удаление элемента по значению)" << endl;
cout << "Текущее множество A: ";
A.print();

try {
    cout << "\nДемонстрация удаления элемента 5:" << endl;
    Set E = A - 5;
    cout << "Исходное A: ";
    A.print();
    cout << "A после A - 5 (новое множество): ";
    E.print();
    cout << "Исходное A не изменилось: ";
    A.print();

    cout << "\nПопытка удалить элемент 99:" << endl;
    Set F = A - 99;
    cout << "Результат: ";
    F.print();
}
catch (int e) {
    cout << "Исключение с кодом: " << e << endl;
}

```

```

cout << "\nТест 6: Исключение при добавлении дубликата" << endl;
try {
    cout << "Попытка добавить 5 в A (5 уже существует): ";
    A.add(5);
}
catch (int e) {
    cout << "Исключение с кодом: " << e << endl;
}

cout << "\nТест 7: Исключение при доступе за границы" << endl;
try {
    cout << "Попытка доступа к A[10]: ";
    cout << A[10] << endl;
}
catch (int e) {
    cout << "Исключение с кодом: " << e << endl;
}

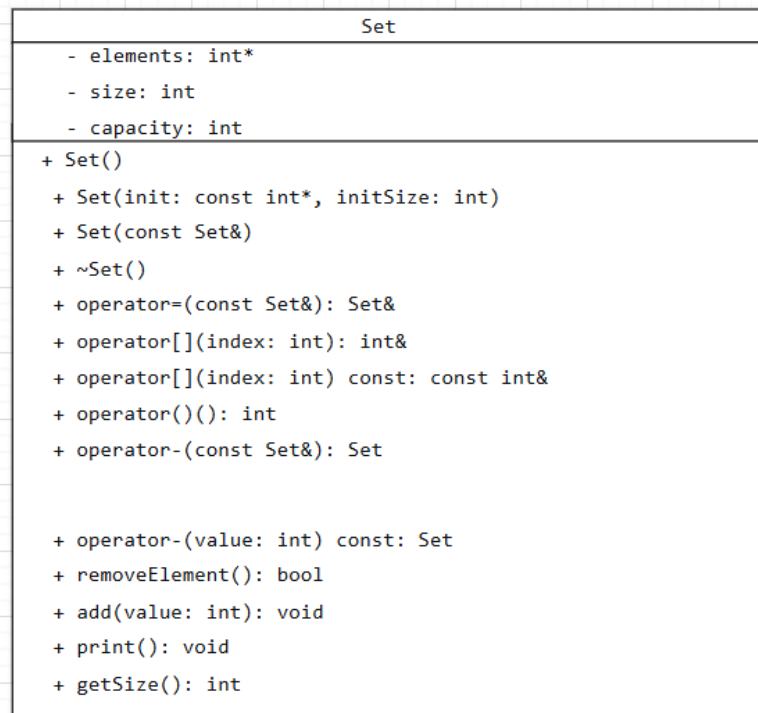
cout << "\nТест 8: Работа с пустым множеством" << endl;
Set empty;
cout << "Пустое множество: ";
empty.print();
cout << "Размер пустого множества: " << empty() << endl;

cout << "\nПРОГРАММА ЗАВЕРШЕНА" << endl;

return 0;
}

```

UML диаграмма



Вывод программы

ТЕСТИРОВАНИЕ КЛАССА МНОЖЕСТВО

Тест 1: Создание множеств

Множество A: { 1, 3, 5, 8, 10 } (размер: 5)

Множество B: { 2, 5, 7, 8 } (размер: 4)

Тест 2: Оператор () - определение размера

Размер A: 5

Размер B: 4

Тест 3: Оператор [] - доступ по индексу

A[0] = 1

A[1] = 3

A[2] = 5

A[3] = 8

A[4] = 10

Тест 4: Оператор - (разность множеств)

A - B = { 1, 3, 10 } (размер: 3)

B - A = { 2, 7 } (размер: 2)

Тест 5: Оператор - (удаление элемента по значению)

Текущее множество A: { 1, 3, 5, 8, 10 } (размер: 5)

Демонстрация удаления элемента 5:

Исходное A: { 1, 3, 5, 8, 10 } (размер: 5)

A после A - 5 (новое множество): { 1, 3, 8, 10 } (размер: 4)

Исходное A не изменилось: { 1, 3, 5, 8, 10 } (размер: 5)

Попытка удалить элемент 99:

Результат: { 1, 3, 5, 8, 10 } (размер: 5)

Тест 6: Исключение при добавлении дубликата

Попытка добавить 5 в A (5 уже существует): Ошибка: элемент 5 уже существует во множестве

Исключение с кодом: 4

Тест 7: Исключение при доступе за границы

Попытка доступа к A[10]: Ошибка: индекс вне диапазона. Размер множества: 5, запрошен индекс: 10

Исключение с кодом: 1

Тест 8: Работа с пустым множеством

Пустое множество: { } (размер: 0)

Размер пустого множества: 0

ПРОГРАММА ЗАВЕРШЕНА

Ответы на вопросы

1. Исключение в C++ — это событие, возникающее при ошибке во время выполнения программы. Исключение представляется объектом любого типа, который передаётся от места возникновения ошибки в обрабатывающий блок.
2. Исключения позволяют разделить вычислительный процесс на нормальное выполнение и обработку ошибок. Достоинства: код обработки ошибок отделён от основного кода; не нужно проверять возвращаемые коды; упрощается поддержка и чтение программы.
3. Для генерации исключительной ситуации используется оператор `throw`.
4. Контролируемый блок — это блок кода, заключённый в `try { }`. Он нужен для того, чтобы перехватывать исключения, возникшие внутри этого блока, и передавать их в блоки `catch`.
5. Секция-ловушка (`catch`) — это блок, следующий за `try`, который перехватывает и обрабатывает исключения определённого типа. Она нужна для выполнения действий при возникновении ошибки.
6. Формы спецификации исключения в `catch`:

- catch (int x) — перехват конкретного типа;
- catch (const char* s) — перехват строки;
- catch (std::exception& e) — перехват стандартного исключения по ссылке;
- catch (...) — перехват любого исключения (ловушка всех исключений).

7. Стандартный класс для создания собственной иерархии исключений — `std::exception` из заголовочного файла `<exception>`.

8. Создать собственную иерархию исключений можно путём наследования от `std::exception` и переопределения виртуального метода `what()`.

```
class MyException : public std::exception
{
public:
    const char* what() const noexcept override
    {
        return "My error message";
    }
};
```

9. Функция `void f1() throw(int, double)`; может генерировать исключения типа `int` и `double`. Генерация любого другого типа приведёт к вызову `unexpected()`.

10. Функция `void f1() throw()`; не может генерировать никакие исключения. При попытке сгенерировать любое исключение вызывается `unexpected()`.

11. Исключение может генерироваться в любом месте программы: внутри функций, методов, конструкторов, деструкторов (не рекомендуется) и в самом блоке `try`.

12. Функция вычисления площади треугольника по формуле Герона.

Вариант 1: без спецификации исключений

```
double area(double a, double b, double c)
```

```
{
    double p = (a + b + c) / 2;
    return sqrt(p * (p - a) * (p - b) * (p - c));
}
```

Вариант 2: со спецификацией throw()

```
double area(double a, double b, double c) throw()
```

```
{
    double p = (a + b + c) / 2;
    return sqrt(p * (p - a) * (p - b) * (p - c));
}
```

Вариант 3: с конкретной спецификацией и стандартным исключением

```
double area(double a, double b, double c)
```

```
{
    if (a <= 0 || b <= 0 || c <= 0)
        throw std::invalid_argument("Стороны должны быть положительными");
    if (a + b <= c || a + c <= b || b + c <= a)
        throw std::invalid_argument("Стороны не образуют треугольник");
    double p = (a + b + c) / 2;
    return sqrt(p * (p - a) * (p - b) * (p - c));
}
```

Вариант 4: с собственным реализованным исключением

```
class TriangleError : public std::exception
```

```
{
    const char* what() const noexcept override
    {
        return "Ошибка треугольника: неверные стороны";
    }
};
```

```
double area(double a, double b, double c)
```

```
{  
    if (a <= 0 || b <= 0 || c <= 0)  
        throw TriangleError();  
    if (a + b <= c || a + c <= b || b + c <= a)  
        throw TriangleError();  
    double p = (a + b + c) / 2;  
    return sqrt(p * (p - a) * (p - b) * (p - c));  
}
```